Peter Sestoft

Java Precisely

Third Edition

The MIT Press
Cambridge, Massachusetts
London, England

© 2016 Massachusetts Institute of Technology

All rights reserved. No part of this book may be reproduced in any form by any electronic or mechanical means (including photocopying, recording, or information storage and retrieval) without permission in writing from the publisher.

MIT Press books may be purchased at special quantity discounts for business or sales promotional use.

This book was set in Times by the author using LaTeX and was printed and bound in the United States of America.

Library of Congress Cataloging-in-Publication Data
Names: Sestoft, Peter.
Title: Java precisely / Peter Sestoft.
Description: Third edition. | Cambridge, MA : The MIT Press, [2016] | Includes bibliographical references and index.
Identifiers: LCCN 2015038522 | ISBN 9780262529075 (pbk. : alk. paper)
Subjects: LCSH: Java (Computer program language)
Classification: LCC QA76.73.J38 S435 2015 | DDC 005.13/3–dc23 LC record available at http://lccn.loc.gov/2015038522

10 9 8 7 6 5 4 3 2 1

Contents

Preface **xi**

Notational Conventions **xii**

1 Running Java: Compilation, Loading, and Execution **2**

2 Names and Reserved Names **2**

3 Java Naming Conventions **2**

4 Comments and Program Layout **2**

5 Types **4**
5.1 Primitive Types . . . 4
5.2 Reference Types . . . 4
5.3 Array Types . . . 4
5.4 Boxing: Wrapping Primitive Types as Reference Types . . . 4
5.5 Subtypes and Compatibility . . . 6
5.6 Signatures and Subsumption . . . 6
5.7 Type Conversion . . . 6

6 Variables, Parameters, Fields, and Scope **8**
6.1 Values Bound to Variables, Parameters, or Fields . . . 8
6.2 Variable Declarations . . . 8
6.3 Scope of Variables, Parameters, and Fields . . . 8

7 Strings **10**
7.1 String Formatting . . . 12

8 Arrays **16**
8.1 Array Creation and Access . . . 16
8.2 Array Initializers . . . 16
8.3 Multidimensional Arrays . . . 18
8.4 The Utility Class Arrays . . . 18

9 Classes **22**
9.1 Class Declarations and Class Bodies . . . 22
9.2 Top-Level Classes, Nested Classes, Member Classes, and Local Classes . . . 22
9.3 Class Modifiers . . . 22
9.4 The Class Modifiers `public`, `final`, `abstract` . . . 24
9.5 Subclasses, Superclasses, Class Hierarchy, Inheritance, and Overriding . . . 24
9.6 Field Declarations in Classes . . . 26
9.7 The Member Access Modifiers `private`, `protected`, `public` . . . 26
9.8 Method Declarations . . . 28
9.9 Parameter Arrays and Variable-Arity Methods . . . 30

9.10 Constructor Declarations . . . 30
9.11 Nested Classes, Member Classes, Local Classes, and Inner Classes . . . 32
9.12 Anonymous Classes . . . 32
9.13 Initializer Blocks, Field Initializers, and Initializers . . . 32

10 Classes and Objects in the Computer **34**
10.1 What Is a Class? . . . 34
10.2 What Is an Object? . . . 34
10.3 Inner Objects . . . 34

11 Expressions **36**
11.1 Table of Expression Forms . . . 36
11.2 Arithmetic Operators . . . 38
11.3 Logical Operators . . . 38
11.4 Bitwise Operators and Shift Operators . . . 38
11.5 Assignment Expressions . . . 40
11.6 Conditional Expressions . . . 40
11.7 Object Creation Expressions . . . 40
11.8 Instance Test Expressions . . . 40
11.9 Field Access Expressions . . . 42
11.10 The Current Object Reference `this` . . . 42
11.11 Type Cast Expression . . . 42
11.12 Method Call Expressions . . . 44
11.13 Lambda Expressions (Java 8.0) . . . 48
11.14 Method Reference Expressions (Java 8.0) . . . 48

12 Statements **52**
12.1 Expression Statements . . . 52
12.2 Block Statements . . . 52
12.3 The Empty Statement . . . 52
12.4 Choice Statements . . . 54
12.5 Loop Statements . . . 56
12.6 Returns, Labeled Statements, Exits, and Exceptions . . . 60
12.7 The Try-with-Resources Statement . . . 64
12.8 The `assert` Statement . . . 64

13 Interfaces **66**
13.1 Interface Declarations . . . 66
13.2 Classes Implementing Interfaces . . . 66
13.3 Default and Static Methods on Interfaces (Java 8.0) . . . 68
13.4 Annotation Type Declarations . . . 68

14 Enum Types **70**

15 Exceptions, Checked and Unchecked **72**

16 Compilation, Source Files, Class Names, and Class Files **74**

17 Packages and Jar Files **74**

18 Mathematical Functions **76**

19 String Builders and String Buffers **78**

20 Threads, Concurrent Execution, and Synchronization **80**
20.1 Threads and Concurrent Execution . . . 80
20.2 Locks and the `synchronized` Statement . . . 82
20.3 Operations on Threads . . . 84
20.4 Operations on Locked Objects . . . 84
20.5 The Java Memory Model and Visibility Across Threads . . . 86

21 Generic Types and Methods **88**
21.1 Generics: Safety, Generality, and Efficiency . . . 88
21.2 Generic Types, Type Parameters, and Type Instances . . . 88
21.3 How Can Type Instances Be Used? . . . 88
21.4 Generic Classes . . . 90
21.5 Constraints on Type Parameters . . . 92
21.6 How Can Type Parameters Be Used? . . . 92
21.7 Generic Interfaces . . . 94
21.8 Generic Methods . . . 96
21.9 Wildcard Type Arguments . . . 98
21.10 The Raw Type . . . 100
21.11 The Implementation of Generic Types and Methods . . . 100

22 Generic Collections and Maps **102**
22.1 Interface Collection<T> . . . 104
22.2 Interface List<T> and Its Implementations LinkedList<T> and ArrayList<T> . . . 105
22.3 Interface Set<T> and Its Implementations HashSet<T> and LinkedHashSet<T> . . . 106
22.4 Interface SortedSet<T> and Implementation TreeSet<T> . . . 106
22.5 Interface Map<K,V> and Implementation HashMap<K,V> . . . 108
22.6 Interface SortedMap<K,V> and Implementation TreeMap<K,V> . . . 110
22.7 Going through a Collection: Interfaces Iterator<T> and Iterable<T> . . . 112
22.8 Interface ListIterator<T> . . . 114
22.9 Equality, Hash Codes, and Comparison . . . 114
22.10 The Comparator<T> Interface . . . 116
22.11 The Utility Class Collections . . . 118
22.12 Choosing the Right Collection Class or Map Class . . . 120

23 Functional Interfaces (Java 8.0) **122**
23.1 Functional Programming . . . 122
23.2 Generic Functional Interfaces . . . 124
23.3 Primitive-Type Specialized Functional Interfaces . . . 124
23.4 Covariance and Contravariance in Functional Interfaces . . . 126
23.5 Interface Function<T,R> . . . 126
23.6 Interface UnaryOperator<T> . . . 126

23.7 Interfaces Predicate<T> and BiPredicate<T,U> . . . 128
23.8 Interfaces Consumer<T> and BiConsumer<T,U> . . . 128
23.9 Interface Supplier<T> . . . 128
23.10 Interface BiFunction<T,U,R> . . . 130
23.11 Interface BinaryOperator<T> . . . 130

24 Streams for Bulk Data (Java 8.0) 132
24.1 Creating Streams . . . 134
24.2 Stream Builders . . . 134
24.3 Methods on Streams . . . 136
24.4 Numeric Streams: DoubleStream, IntStream, and LongStream . . . 140
24.5 Summary Statistics for Numeric Streams . . . 140
24.6 Collectors on Streams . . . 142

25 Class Optional<T> (Java 8.0) 146

26 Input and Output 148
26.1 Creating an IO Stream from Another One . . . 149
26.2 Kinds of Input and Output Methods . . . 150
26.3 Imports, Exceptions, Thread Safety . . . 150
26.4 Sequential Character Input: Readers . . . 152
26.5 Sequential Character Output: Writers . . . 153
26.6 Printing Primitive Data to a Character Stream: PrintWriter . . . 154
26.7 The Appendable Interface and the CharSequence Interface . . . 154
26.8 Reading Primitive Data from a Character Stream: StreamTokenizer . . . 156
26.9 Sequential Byte Input: InputStream . . . 158
26.10 Sequential Byte Output: OutputStream . . . 159
26.11 Binary Input-Output of Primitive Data: DataInput and DataOutput . . . 160
26.12 Serialization of Objects: ObjectInput and ObjectOutput . . . 162
26.13 Buffered Input and Output . . . 164
26.14 Random Access Files: RandomAccessFile . . . 166
26.15 Files, Directories, and File Descriptors . . . 168
26.16 Thread Communication: PipedInputStream and PipedOutputStream . . . 168
26.17 Socket Communication . . . 170

27 Reflection 172
27.1 Reflective Use of Types: The Class<T> Class . . . 172
27.2 Reflection: The Field Class . . . 174
27.3 Reflection: The Method Class and the Constructor<T> Class . . . 174
27.4 Exceptions Thrown When Using Reflection . . . 174

28 Metadata Annotations 176

29 What Is New in Java 8.0 178

References 180

Index **181**

24 Streams for Bulk Data (Java 8.0)

A *stream* represents a number of data elements, or bulk data, though usually not explicitly stored in the manner of an array or collection. A stream is typically processed by a pipeline, built from a *stream generator*, several *intermediate stream operations*, and a single *terminal stream operation*. A stream generator produces a stream, an intermediate operation consumes and produces a stream, and a terminal operation only consumes a stream. This approach supports mostly functional data processing and in particular enables parallel computation in a painless and safe manner.

A stream is often lazily generated and lazily processed, and the creation and processing of stream elements are driven by the terminal operation's pull (demand) at the end of the pipeline rather than the stream generator's push at the beginning of the pipeline. So only a small fraction of the stream's elements are actually stored in memory at any given time. In fact, a lazily created stream may be infinite, although some operations such as `count()` would never terminate on such a stream. Moreover, some intermediate operations, such as consecutive `map` transformations, may be fused, so that intermediate results are actually never stored at all. This makes functional stream pipelines very fast, sometimes faster than the "obviously best" imperative code; see example 167.

For instance, while it seems that `xs.map(f).map(g)` will perform two traversals of the stream `xs`, transforming the elements first by function `f` and then by function `g`, this can be implemented behind the scenes by a single traversal `xs.map(f.andThen(g))` applying the composition of `f` and `g`; see example 156.

A stream may be *sequential* or *parallel*. On a sequential stream, intermediate and terminal operations will be performed sequentially, on a single thread. On a parallel stream, intermediate and terminal operations may be performed in parallel on multiple threads.

A stream may be *ordered* or *unordered*. For an ordered stream, intermediate operations such as `filter(p)` and `map(f)` respect the element order and produce a stream with elements in the expected order, even if the stream is parallel. For an unordered stream, there is no such guarantee, so some parallel intermediate operations may be more efficient on unordered streams.

Also note that on a parallel stream, even an ordered one, there is no guaranteed order in which the functional arguments `f` and `p` are applied to the stream's elements: the only guarantee is that the resulting stream's elements appear in the expected order.

For example, the result of `IntStream.range(0,5).parallel().map(x -> x*2).toArray()` is guaranteed to be an array containing the elements `[0, 2, 4, 6, 8]`, but the function `(x -> x*2)` may be applied to element `3` before element `2`, or vice versa, or at the exact same time.

Regardless whether the stream is ordered or unordered, the terminal operation `forEach` does not respect the element order for parallel streams.

Since operations on a parallel stream may be evaluated on multiple threads and in an unpredictable order, the functions passed to stream operations should be *stateless*: they must not depend on any state that may change during the pipeline computation, not even state internal to the function. Example 172 uses a predicate that is not stateless and therefore does not work on parallel streams.

A stream's elements can be consumed only once; trying to use a stream twice will throw IllegalStateException. In this respect Java streams are very different from Haskell's lazy lists, for instance.

When a stream is created from a source such as an array, collection, or file, that source must not be modified while the stream is being used; otherwise the results are unpredictable, and exceptions may be thrown. The functions passed to stream operations should be *non-interfering*: they must not modify the stream's source.

Example 164 Creating Finite Streams
A finite sequential stream can be created by enumerating its elements, from an array, or from a collection such as a set:

```
IntStream is = IntStream.of(2, 3, 5, 7, 11, 13);

String[] a = { "Hoover", "Roosevelt", "Truman", "Eisenhower", "Kennedy" };
Stream<String> presidents = Arrays.stream(a);

Collection<String> coll = new HashSet<String>();
coll.add("Denmark"); coll.add("Norway"); coll.add("Sweden");
Stream<String> countries = coll.stream();
```

Example 165 Creating an Infinite Stream of Prime Numbers
One can create an infinite sequential stream `nats` of the natural numbers by starting with 0 and adding 1 to each successive element, using the `iterate` method. One can then create an infinite stream `primes` of prime numbers (a natural number divisible only by 1 and itself) by filtering the stream of natural numbers. Simple and efficient.

```
IntStream nats = IntStream.iterate(0, x -> x+1);
IntStream primes = nats.filter(x -> isPrime(x));
```

Example 166 Creating a Finite Stream of Prime Numbers
The simplest way to create a finite sequential stream of the first `n` prime numbers is to create an infinite stream, as in example 165, and then `limit` it to the first `n` elements:

```
public static IntStream primes2(int n) {
  return IntStream.iterate(0, x -> x+1).filter(x -> isPrime(x)).limit(n);
}
```

Example 167 Counting Prime Numbers: Sequential Stream, Imperative Loop, and Parallel Stream
One can count the prime numbers less than 10 million by this stream pipeline. It generates the integers between 0 and 10 million, tests whether each of them is a prime, throws away the non-primes, and counts the rest:

```
IntStream.range(0, 10_000_000).filter(i -> isPrime(i)).count()
```

Since the numbers are lazily generated, this uses very little memory, and in fact the above stream expression is just as fast as a classic efficient-looking imperative loop:

```
int count = 0;
for (int i=0; i<10_000_000; i++)
  if (isPrime(i))
    count++;
```

The real advantage of the stream pipeline is that it is trivial to parallelize: just insert `.parallel()`, then the prime number testing and counting will exploit any available parallel processor cores. The parallelized version below is four times faster than the imperative loop on a 4-core laptop, and sixteen times faster than the imperative loop on a 32-core server. Parallelizing the imperative loop is vastly more cumbersome and not more efficient.

```
IntStream.range(0, 10_000_000).parallel().filter(i -> isPrime(i)).count()
```

24.1 Creating Streams

There are many ways to create a stream:

- By explicit enumeration of elements, using the variable-arity static method `Stream.of(T ...)`, which creates a sequential ordered stream, or `Stream.empty()`, which creates a stream with no elements.
- From an array, using the static methods `Arrays.stream(T[])` and `Arrays.stream(T[],from,to)` from class Arrays (section 8.4), both of which create a sequential Stream<T>. There are primitive-type specialized overloads `Arrays.stream(double[])` and `Arrays.stream(double[],from,to)` that create a sequential DoubleStream, and similar ones for IntStream and LongStream.
- From a collection, using the Collection<T> default method `coll.stream()`, which creates a sequential Stream<T>, or default method `coll.parallelStream()`, which creates a possibly parallel Stream<T>.
- By static generator methods on Stream<T> such as `iterate(x0, f)` and `generate(supp)`, or from IntStream's and LongStream's static methods `range(from,to)` and `rangeClosed(from,to)`.
- By imperative generation of stream elements, using a stream builder; see section 24.2.
- From a BitSet (package `java.util`) using method `stream()`, which returns an IntStream of the numbers in the set; see examples 175 and 176.
- From a random number generator of class Random, using methods `ints(n)`, `ints()`, `ints(a,b)`, or `ints(n,a,b)`, which produce an IntStream of `n` random integers, or infinitely many random integers, possibly limited to the range `a..(b-1)`; or using corresponding methods called `doubles` and `longs` to generate a DoubleStream or LongStream.
- From a BufferedReader using the `lines()` method, which generates a Stream<String>; see example 170.

24.2 Stream Builders

The Stream.Builder<T> interface from `java.util.stream` extends the Consumer<T> interface (section 23.8) and can be used to build a sequential stream using imperative programming. A stream builder computes the stream elements eagerly, so it cannot create an infinite stream, and it may do more work than necessary in case only some of the generated elements are ever consumed.

The Stream.Builder<T> interface has two abstract methods and a default one:

- `void accept(T x)` adds an element to the stream being built.
- `default Stream.Builder<T> add(T x)` works exactly as `accept(x)` but in addition returns the stream builder to allow chained calls, as in `sb.add(2).add(3).add(5)`.
- `Stream<T> build()` builds the stream and moves the stream builder to the built state. After this call, any call to `accept` or `add` will throw IllegalStateException.

The primitive-type specialized interfaces {Double,Int,Long}Stream.Builder extend the DoubleConsumer, IntConsumer, and LongConsumer interfaces (section 23.8) and have the same methods as listed above, with corresponding specialized argument and return types.

Example 168 Using a Stream Builder to Create a Stream of Prime Numbers
One can use a stream builder to create a stream of the first `n` prime numbers as shown below. This will compute the prime numbers eagerly, that is, before anything is consumed from the stream. The functional way to generate such a stream is much more elegant, more efficient, and parallelizable if desired; see example 166.

```
public static IntStream primes4(int n) {
  IntStream.Builder isb = IntStream.builder();
  int p = 2, count = 0;
  while (count < n) {
    if (isPrime(p)) {
      isb.accept(p);
      count++;
    }
    p++;
  }
  return isb.build();
}
```

Example 169 Using a Stream Builder to Collect Pattern Matches
The regular expression `urlPattern` below matches a link `a href="link"` in a web page. Using a stream builder, one can create a stream of Links whose elements are pairs (`url`, `link`) of the web page `url` and each `link` found in the web page. The stream is built eagerly: all links must be found before the stream can be used.

```
public static Stream<Link> scanLinks(Webpage page) {
  Matcher urlMatcher = urlPattern.matcher(page.contents);
  Stream.Builder<Link> links = Stream.<Link>builder();
  while (urlMatcher.find()) {
    String link = urlMatcher.group(1);
    links.accept(new Link(page.url, link));
  }
  return links.build();
}
final static Pattern urlPattern = Pattern.compile("a href=\"(\\p{Graph}*)\"");
```

Example 170 Reading a Stream of Lines from a BufferedReader
A stream of the text lines making up a web page can be obtained by reading the web page through a BufferedReader (section 26.13) and creating a lazy sequential Stream<String> from the web page. The consumer of the stream determines how much of the web page will actually be read via the network. It might be tempting to close the BufferedReader before returning the stream of lines, but this is wrong and likely would throw an UncheckedIOException. The BufferedReader will be in use as long as lines are consumed from the stream, and only when the stream gets closed may the reader and input stream be closed too.

```
public static Stream<String> getPageLines(String url) {
  try {
    InputStreamReader isr = new InputStreamReader(new URL(url).openStream());
    BufferedReader reader = new BufferedReader(isr);
    return reader.lines();
  } catch (IOException exn) {
    return Stream.<String>empty();
  }
}
```

24.3 Methods on Streams

Interface Stream<T> from package `java.util.stream` has methods for creating, further processing, or consuming streams. In the descriptions below, the elements of a stream are denoted `x1`, `x2`, ..., and in general `xi`. For brevity we have simplified some types in the method signatures, as explained in section 23.4.

- `boolean allMatch(Predicate<T> p)` returns true if `p.test(xi)` is true for all elements, else false.
- `boolean anyMatch(Predicate<T> p)` returns true if `p.test(xi)` is true for some element, else false.
- `static <T> Stream.Builder<T> builder()` returns a builder for a Stream<T>; see section 24.2.
- `void close()` closes this stream, causing all close handlers for this stream pipeline to be called.
- `R collect(Collector<? super T,A,R> coltor)` performs a mutable reduction operation on the stream using the collector; see section 24.6. Interfaces {Double,Int,Long}Stream do not have this method.
- `R collect(Supplier<R> supp, BiConsumer<R,T> accumulate, BiConsumer<R,R> combine)` performs a mutable reduction operation using the collector components; see section 24.6 and example 186.
- `static <T> Stream<T> concat(Stream<? extends T> xs, Stream<? extends T> ys)` creates a lazy stream whose elements are the elements of `xs` followed by the elements of `ys`.
- `long count()` returns the number of elements in this stream.
- `Stream<T> distinct()` returns a stream without duplicate elements, as determined by `x.equals(y)`.
- `static <T> Stream<T> empty()` returns an empty sequential stream.
- `Stream<T> filter(Predicate<T> p)` returns a stream of the `xi` for which `p.test(xi)` is true.
- `Optional<T> findAny()` returns an Optional containing some element from the stream if non-empty; otherwise returns an empty Optional (see section 25).
- `Optional<T> findFirst()` returns an Optional containing the first element from the stream if non-empty; otherwise returns an empty Optional.
- `<R> Stream<R> flatMap(Function<T,Stream<R>> f)` returns a stream whose elements result from computing `f.apply(x1)`, `f.apply(x2)`, ... to produce a sequence of streams, and flattening the resulting streams into one. The primitive-type specialized methods `flatMapTo{Double,Int,Long}` correspondingly produce streams of type {Double,Int,Long}Stream.
- `void forEach(Consumer<T> cons)` performs `cons.accept(xi)` on the elements `xi` of this stream.
- `void forEachOrdered(Consumer<T> cons)` performs `cons.accept(xi)` on the elements `xi` of this stream, in encounter order.
- `static <T> Stream<T> generate(Supplier<T> supp)` returns an infinite sequential unordered stream resulting from the call sequence `supp.get()`, `supp.get()`, ... where `supp` is possibly stateful. Note that unlike for iterators (section 22.7), there is no way to indicate end-of-stream.
- `boolean isParallel()` returns true if this is a parallel stream, otherwise false.
- `static <T> Stream<T> iterate(T x0, UnaryOperator<T> f)` returns an infinite sequential ordered stream whose elements `ri` are `r0` = `x0`, `r1` = `f.apply(r0)`, `r2` = `f.apply(r1)`, ...
- `Iterator<T> iterator()` returns an iterator for the elements of this stream.
- `Stream<T> limit(long n)` returns a stream consisting of at most the first `n` elements of this stream.

Example 171 Using Stream Methods to Find and Print Web Page Links
This example reads web pages from the net, scans the first 200 lines of each web page for links, discards duplicate links and prints the unique ones, using streams and functional programming to cleanly separate these tasks. Method `getPage` from example 181 returns a Webpage object consisting of a URL and a string holding the first 200 lines of the page contents. Method `scanLinks` from example 169 scans a (partial) web page for hyperlinks and returns a stream of Links. The stream method `flatMap` calls `scanLinks` on many web pages to obtain many Link streams and then flattens all those into a single Link stream. The stream method `distinct` discards duplicates from the Link stream. The stream method `forEach` prints the links as they are produced.

```
String[] allUrls = { "http://www.itu.dk", ... };

Stream<String> urls = Stream.<String>of(allUrls);
Stream<Webpage> pages = urls.map(url -> getPage(url, 200));
Stream<Link> links = pages.flatMap(page -> scanLinks(page));
Stream<Link> uniqueLinks = links.distinct();
uniqueLinks.forEach(System.out::println);  // Calls Link.toString()
```

Example 172 Checking Sortedness of a Sequential Stream
To check whether a sequential ordered stream is sorted, one can use the stream method `allMatch` together with a stateful predicate as shown below. Each application of the predicate `x -> { ... }` compares a stream element `x` to its predecessor. The singleton array `last[0]` holds that predecessor; we cannot use a plain `int` variable for that purpose because variables captured in a Java lambda expression must be effectively final, that is, immutable (section 9.11). This method does not work on a parallel stream because the predicate is stateful.

```
static boolean isSorted2(IntStream xs) {
  final int[] last = { Integer.MIN_VALUE };
  return xs.allMatch(x -> { int old = last[0]; last[0] = x; return old <= x; } );
}
```

Example 173 Making a Stream of English Numerals
Using function `toEnglish` from example 158 one can create a (practically) infinite stream of the English numerals "zero", "one", "two", ..., "thirteen million nine hundred eighty-nine thousand four hundred twenty-two", and so on. One can also generate a stream of "logorithms", where the logorithm of a number `n` is the number of letters in its numeral (we believe this tongue-in-cheek concept is attributable to Martin Gardner).

```
Stream<String> numerals
  = LongStream.iterate(0, x -> x+1).mapToObj(Numerals::toEnglish);
IntStream logorithms = numerals.mapToInt(String::length);
System.out.println(logorithms.limit(1_000_000).max());
```

Example 174 Versatility of Streams
Streams are very versatile. For instance, if we can lazily generate a stream of solutions to the 8-queens problem (example 176), then we can later decide whether we want to print all solutions, the number of solutions, the first 20 solutions, or an arbitrary solution, as shown below. With a more imperative approach, we would typically have to decide beforehand how to use the results.

```
queens(8).forEach(System.out::println);
System.out.println(queens(8).count());
queens(8).limit(20).forEach(System.out::println);
System.out.println(queens(8).findAny());
```

Methods on interface Stream<T> continued:

- `Stream<R> map(Function<T,R> f)` returns a stream with elements `f.apply(x1)`, `f.apply(x2)`, ... The primitive-type specialized methods `mapTo{Double,Int,Long}` correspondingly produce streams of type {Double,Int,Long}Stream.
- `Optional<T> max(Comparator<T> cmp)` returns the stream's maximal element according to `cmp`, or an absent Optional if there are no elements.
- `Optional<T> min(Comparator<T> cmp)` returns the stream's minimal element according to `cmp`, or an absent Optional if there are no elements.
- `boolean noneMatch(Predicate<T> p)` returns true if `p.test(xi)` is false for all elements, else false.
- `static <T> Stream<T> of(T... vs)` returns a sequential ordered stream whose elements are the `vs`.
- `static <T> Stream<T> of(T x)` returns a sequential Stream containing the single element `x`.
- `Stream<T> onClose(Runnable handler)` returns a stream with the same elements but an additional close handler. When closing the stream, the close handlers are executed in the order they were added.
- `Stream<T> parallel()` returns a parallel stream with the same elements as this stream.
- `Stream<T> peek(Consumer<T> cons)` returns a stream consisting of the same elements `x1`, `x2`, ... as this stream, additionally performing actions `cons.accept(x1)`, `cons.accept(x2)`, ... as the elements are being consumed from the resulting stream. Use it for debugging purposes only.
- `Optional<T> reduce(BinaryOperator<T> op)` computes the reduction of the stream's elements using the associative operator `op`. More precisely, writing `op.apply(x,y)` as infix `x`⊗`y`, returns an Optional containing the value `x1`⊗`x2`⊗...⊗`xn`, computed in some order, if the stream is non-empty; otherwise returns an empty Optional.
- `T reduce(T x0, BinaryOperator<T> op)` computes the reduction of `x0` and the stream's elements using the associative operator `op`. More precisely, writing `op.apply(x,y)` as infix `x`⊗`y`, the method returns `x0`⊗`x1`⊗`x2`⊗...⊗`xn`, computed in some order.
- `U reduce(U r0, BiFunction<U,T,U> op, BinaryOperator<U> comb)` computes the reduction of the stream's elements, using the provided identity `r0`, accumulation function `op`, and combiner `comb`. More precisely, writing `op.apply(r,x)` as infix `r`⊗`x` and writing the combiner `comp.apply(r,s)` as infix `r`⊕`s`, returns `(r0`⊗`x11`⊗...⊗`x1m)` ⊕ ... ⊕ `(r0`⊗`xk1`⊗...⊗`xkm)`, computed in some order, where the `xij` represents some partitioning of the stream's elements into segments. It must hold that `r0`⊗`x` equals `x`, that `r`⊕`(r0`⊗`x)` equals `r`⊗`x` for all `x` and `r`, and ⊕ must be associative. The primitive-type specialized {Double,Int,Long}Stream interfaces do not have this overload.
- `Stream<T> sequential()` returns a sequential stream with the same elements as this stream.
- `Stream<T> skip(long n)` returns a stream of the remaining elements after discarding the `n` first ones.
- `Stream<T> sorted()` returns a stream consisting of the elements of this stream, sorted in natural order.
- `Stream<T> sorted(Comparator<T> cmp)` returns a stream consisting of the elements, sorted by `cmp`.
- `Spliterator<T> spliterator()` returns a spliterator for the elements of this stream.
- `Object[] toArray()` returns an array containing the elements of this stream.
- `T[] toArray(IntFunction<T[]> alloc)` returns an array containing the elements of this stream, using `alloc.apply(n)` to allocate the returned array as well as any intermediate arrays.
- `Stream<T> unordered()` returns an unordered stream with the same elements as this stream.

Example 175 Generating a Stream of Permutations
The stream of all permutations of n numbers $0 \ldots (n-1)$ can be generated by maintaining a partially generated permutation as an integer list `tail`, and a set `todo` of the numbers not yet used in the permutation. If `todo` is empty, `tail` is a permutation of all n numbers. Otherwise recursively generate those permutations that can be obtained by removing an element `r` from `todo` and putting it in front of `tail`. To create all n-permutations, start with an empty `tail`, and a `todo` set containing the numbers $0 \ldots (n-1)$.

Class IntList represents immutable integer lists; see example 182. The `boxed()` operation turns an IntStream into a Stream<Integer> so one can apply `flatMap` to obtain a Stream<IntList>; the `flatMap` method on IntStream produces only IntStreams. The call `minus(todo, r)` returns a new BitSet with `r` removed.

```
public static Stream<IntList> perms(BitSet todo, IntList tail) {
  if (todo.isEmpty())
    return Stream.of(tail);
  else
    return todo.stream().boxed().flatMap(r -> perms(minus(todo, r), new IntList(r, tail)));
}
public static Stream<IntList> perms(int n) {
  BitSet todo = new BitSet(n); todo.flip(0, n); return perms(todo, null);
}
```

Example 176 Generating a Stream of Solutions to the n-Queens Problem
We now augment the permutation generator (example 175) to generate solutions to the n-queens problem: How to place n queens on an n-by-n chessboard so all queens are safe from each other. A 3-permutation such as $[1,0,2]$ can be considered a safe placement of 3 rooks on a 3-by-3 chessboard, in rows 1, 0, and 2 of columns 0, 1, and 2. So a solution to the n-queens problem is a permutation further constrained by considering diagonals: Filter away those `r` values from `todo` that, if put in front of `tail`, would constitute an unsafe queen's position that could attack some queen in the columns represented by `tail`.

This solution is simple, quite fast, versatile (example 174), and trivial to parallelize because all operations are purely functional. Putting `.parallel()` after `filter` gives a speed-up of 3.5 on a 4-core i7 processor.

```
public static Stream<IntList> queens(BitSet todo, IntList tail) {
  if (todo.isEmpty())
    return Stream.of(tail);
  else
    return todo.stream()
      .filter(r -> safe(r, tail)).boxed()             // could use .parallel() here
      .flatMap(r -> queens(minus(todo, r), new IntList(r, tail)));
}
public static boolean safe(int mid, IntList tail) { return safe(mid+1, mid-1, tail); }
public static boolean safe(int d1, int d2, IntList tail) {
  return tail==null || d1!=tail.item && d2!=tail.item && safe(d1+1, d2-1, tail.next);
}
```

Example 177 A Stream Can Be Consumed Only Once
A stream can be consumed only once, so one cannot find the standard deviation of a DoubleStream `ds` by separately computing its mean and the sum of its squares; instead compute both in one traversal, as in example 180.

```
DoubleSummaryStatistics stats = ds.summaryStatistics();
// Fails with IllegalStateException: stream has already been operated upon or closed:
double sqsum = ds.map(x -> x*x).sum();
double sdev = Math.sqrt(sqsum/stats.getCount() - stats.getAverage()*stats.getAverage());
```

24.4 Numeric Streams: DoubleStream, IntStream, and LongStream

Numeric streams may be represented by primitive-type specialized interfaces {Double,Int,Long}Stream for efficiency; see section 23.3. They have additional methods `average()`, `max()`, `min()`, `sum()`, and `mapToObj`. The argument and result types of their Stream<T> methods are appropriately primitive-type specialized. For instance, the `iterator()` methods return PrimitiveIterator.Of{Double,Int,Long} objects (section 22.7), and the `generate` method's signatures in IntStream and Stream<T> look like this:

```
static IntStream generate(IntSupplier supp)
static Stream<T> generate(Supplier<T> supp)
```

In addition to the general stream methods (section 24.3), DoubleStream has these methods:

- `Stream<Double> boxed()` returns a stream of this stream's elements, each boxed as a Double object.
- `DoubleSummaryStatistics summaryStatistics()` returns statistics for this stream; see section 24.5.

In addition to the general stream methods (section 24.3), IntStream has these methods:

- `DoubleStream asDoubleStream()` returns a stream of this stream's elements converted to `double`.
- `LongStream asLongStream()` returns a stream of this stream's elements converted to `long`.
- `Stream<Integer> boxed()` returns a stream of this stream's elements, each boxed as an Integer object.
- `static IntStream range(int a, int b)` returns the `int` stream `[a..(b-1)]`, empty if `a>=b`.
- `static IntStream rangeClosed(int a, int b)` returns the `int` stream `[a..b]`, empty if `a>b`.
- `IntSummaryStatistics summaryStatistics()` returns statistics for this stream; see section 24.5.

In addition to the general stream methods (section 24.3), LongStream has these methods:

- `DoubleStream asDoubleStream()` returns a stream of this stream's elements converted to `double`.
- `Stream<Long> boxed()` returns a stream of this stream's elements, each boxed as a Long object.
- `static LongStream range(long a, long b)` returns the `long` stream `[a..(b-1)]`, empty if `a>=b`.
- `static LongStream rangeClosed(long a, long b)` returns `long` stream `[a..b]`, empty if `a>b`.
- `LongSummaryStatistics summaryStatistics()` returns statistics for this stream; see section 24.5.

24.5 Summary Statistics for Numeric Streams

The classes {Double,Int,Long}SummaryStatistics from package `java.util` represent summary statistics of a numeric stream. The classes have `get` methods that return count, min, max, sum, and average (mean); see example 178. The min, max, and sum have the same type as the stream elements, except that the sum of an IntStream is a `long`. The average is always a `double`.

Class DoubleSummaryStatistics implements the DoubleConsumer interface and, in addition to the `get` methods shown in example 178, has these methods to collect the statistics:

- `void accept(double x)` records value `x` in the summary information.
- `void combine(DoubleSummaryStatistics other)` combines the `other` statistics into this one.

The IntSummaryStatistics and LongSummaryStatistics classes have corresponding methods. These methods can be passed to a stream's `collect` method to compute the statistics (example 179), and they can be overridden to collect more comprehensive statistics (example 180).

Example 178 Summary Statistics for Numeric Streams
The summary statistics for a `double` stream can be computed and printed like this, with the printed output inserted as a comment:

```
DoubleStream ds = DoubleStream.of(2, 4, 4, 4, 5, 5, 7, 9);
DoubleSummaryStatistics stats = ds.summaryStatistics();
System.out.printf("count=%d, min=%g, max=%g, sum=%g, mean=%g%n",
                  stats.getCount(), stats.getMin(), stats.getMax(),
                  stats.getSum(), stats.getAverage());
// count=8, min=2.00000, max=9.00000, sum=40.0000, mean=5.00000
```

Example 179 Computing Summary Statistics Using Collector Functions
The DoubleSummaryStatistics object `stats` computed in example 178 can equivalently be computed like this, using the collector components (section 24.6) of the DoubleSummaryStatistics class:

```
DoubleSummaryStatistics stats
  = ds.collect(DoubleSummaryStatistics::new,
               DoubleSummaryStatistics::accept,
               DoubleSummaryStatistics::combine);
```

Example 180 Extending Double Summary Statistics with Standard Deviation
By creating a subclass BetterDoubleStatistics of the DoubleSummaryStatistics class, one can also compute the standard deviation in the same traversal of a stream of `double`s. Note that this cannot be done by multiple traversals (first compute the usual summary statistics, then compute the sum of squares of the stream) because a stream can be consumed only once; see example 177.

The BetterDoubleStatistics class computes the sum of squares in addition to whatever is done by superclass DoubleSummaryStatistics and adds a method `getSdev` to compute the standard deviation afterwards:

```
class BetterDoubleStatistics extends DoubleSummaryStatistics {
  private double sqsum = 0.0;
  @Override
  public void accept(double d) {
    super.accept(d);
    sqsum += d * d;
  }
  public void combine(BetterDoubleStatistics other) {
    super.combine(other);
    sqsum += other.sqsum;
  }
  public double getSdev() {
    double mean = getAverage();
    return Math.sqrt(sqsum/getCount() - mean*mean);
  }
}
...
DoubleStream ds = DoubleStream.of(2, 4, 4, 4, 5, 5, 7, 9);
BetterDoubleStatistics stats
  = ds.collect(BetterDoubleStatistics::new,
               BetterDoubleStatistics::accept,
               BetterDoubleStatistics::combine);
// count=8, min=2.00000, max=9.00000, sum=40.0000, mean=5.00000, sdev=2.00000
```

24.6 Collectors on Streams

In some cases, it is difficult to make the functional stream reduce operations efficient enough. This holds, for instance, for massive string concatenation (where repeated use of `s1+s2` has quadratic execution time); for creating a collection, list, or set from a stream; and for various grouping and binning operations. In those cases, a *mutable reduction operation* using a so-called collector may be more efficient. However, functional reductions should be preferred where possible, because the mutable reduction operations are easier to get wrong and often see much less speed-up (or even considerable slow-down) on parallel streams than the functional operations.

A collector `coltor` is an instance of interface Collector<T,A,R> that can process a stream `xs` of type Stream<T>, using an internal accumulator of type A, and producing a result of type R.

Stream method `xs.collect(coltor)` applies the collector to the stream `xs`, performing the mutable reduction operation and returning a result of type R. Utility class Collectors in package `java.util.stream` defines many useful collectors, listed below.

Stream method `xs.collect(Supplier<R> supp, BiConsumer<R,T> accu, BiConsumer<R,R> comb)` supports custom mutable reduction operations, producing a final result of type R. Function `supp()` generates a result container; function `accu(rc, x)` is called to process stream element `x` and add it to result container `rc`; and function `comb(rc1,rc2)` is called to combine the state of result container `rc2` into `rc1`; see examples 179, 180, and 186.

Some more advanced features of collectors are not described in this book; see the Java class library documentation [2].

Below we list the static methods in class Collectors that produce often used collectors. For readability we have simplified some of the wildcard types in the signatures, as described in section 23.4. We use the parameter names `coltor` for collector, `fin` for finisher, `rc` for result container, `comb` for combiner, `cons` for consumer, and `cfier` for classifier.

- `Collector<T,?,Double> averagingDouble(ToDoubleFunction<T> f)` computes the arithmetic mean, or average, of `f.apply(xi)`. There are similarly named methods for Int and Long.
- `Collector<T,A,RR> collectingAndThen(Collector<T,A,R> coltor, Function<R,RR> fin)` collects by `coltor` and then applies finisher `fin` to the result container.
- `Collector<T,?,Long> counting()` counts the number of elements.
- `Collector<T,?,Map<K,List<T>>> groupingBy(Function<T,K> cfier)` groups elements `xi` into lists by the value of key `cfier.apply(xi)`.
- `Collector<T,?,Map<K,D>> groupingBy(Function<T,K> cfier, Collector<? super T,A,D> coltor)` groups elements `xi` into lists by the value of key `cfier.apply(xi)`, then performs reduction operation `coltor` on the set of values `xi` associated with each key.

  There is also an overload with an additional argument of type `Supplier<Map<K,D>>` to produce the map used. There are concurrent versions of these methods also, named `groupingByConcurrent`; these produce a ConcurrentMap.

  Two `partitioningBy` methods work like the methods above but take a Predicate<T> instead of a Function<T,K> and produce a map whose only keys are `true` and `false`.

Example 181 Using a Collector to Join Lines into a Page
Method `getPageLines` from example 170 produces a lazy stream of the lines of a web page. We can join the first `maxLines` lines into a single string using the stream methods `limit` and `collect`, where the latter is applied to the predefined `joining` collector that efficiently joins strings.

```
public static Webpage getPage(String url, int maxLines) {
  String contents =
    getPageLines(url).limit(maxLines).collect(Collectors.joining());
  return new Webpage(url, contents);
}
```

Example 182 Using a Collector and IntStream to Print an Integer List
Class IntList is used in examples 175 and 176 to represent immutable integer lists, which we would like to print in the format `[1, 3, 0, 2]` within square brackets and with comma-separated numbers. We could cleverly define `toString` using a StringBuilder, but a simpler and more general idea is to define a method `stream` to convert IntList to IntStream and then use a predefined collector to format the IntStream as a string.

```
class IntList {
  public final int item;
  public final IntList next;
  public IntList(int item, IntList next) { this.item = item; this.next = next; }
  public static IntStream stream(IntList xs) {
    IntStream.Builder sb = IntStream.builder();
    while (xs != null) {
      sb.accept(xs.item);
      xs = xs.next;
    }
    return sb.build();
  }
  public String toString() {
    return stream(this).mapToObj(String::valueOf).collect(Collectors.joining(",", "[", "]"));
  }
}
```

Example 183 Using Stream Functions to Generate a van der Corput Sequence
A van der Corput sequence is an infinite sequence $\frac{1}{2}, \frac{1}{4}, \frac{3}{4}, \frac{1}{8}, \frac{5}{8}, \frac{3}{8}, \frac{7}{8}, \ldots$ that is dense in the interval $[0,1]$ and evenly distributed over it. The infinite sequence is typically used in (financial) simulations. A 2 billion element approximation of the sequence may be generated lazily using stream functions: for every bit count `b` in range $1 \ldots 31$ and for every `i` in range $2^{b-1} \ldots 2^b - 1$, compute the bit-reversal of integer `i` and divide by 2^b.

Example 186 uses collectors to test that the generated numbers are evenly distributed over $[0,1]$; and example 24 uses array sort to show that they are dense in $[0,1]$.

```
public static DoubleStream vanDerCorput() {
  return IntStream.range(1, 31).asDoubleStream().flatMap(b -> bitReversedRange((int)b));
}

private static DoubleStream bitReversedRange(int b) {
  final long bp = Math.round(Math.pow(2, b));
  return LongStream.range(bp/2, bp).mapToDouble(i -> (double)(bitReverse((int)i)>>>(32-b))/bp);
}
private static int bitReverse(int i) { ... /* reverse the bits in i */ ...}
```

Static methods on class Collectors that generate collectors continued:

- `Collector<CharSequence,?,String> joining()` concatenates the input elements `xi` into a string.
- `Collector<CharSequence,?,String> joining(CharSequence delim)` concatenates the input elements `xi`, separated by `delim`, into a string.
- `Collector<CharSequence,?,String> joining(CharSequence delim, CharSequence pre, CharSequence suf)` concatenates the input elements `xi`, separated by `delim`, into a string starting with `pre` and ending with `suf`.
- `Collector<T,?,R> mapping(Function<T,U> f, Collector<? super U,A,R> coltor)` applies `f` to each element `xi` and then uses `coltor` to perform a reduction of the `f.apply(xi)` values.
- `Collector<T,?,Optional<T>> maxBy(Comparator<T> cmp)` produces an optional maximal element according to `cmp`, or absent if no elements.
- `Collector<T,?,Optional<T>> minBy(Comparator<T> cmp)` produces an optional minimal element according to `cmp`, or absent if no elements.
- `Collector<T,?,Optional<T>> reducing(BinaryOperator<T> op)` performs a reduction of the elements using `op`. More precisely, writing `op.apply(x,y)` as infix `x`⊗`y`, returns an Optional containing the value `x1`⊗`x2`⊗...⊗`xn` if the stream is non-empty; otherwise returns an empty Optional.
- `Collector<T,?,T> reducing(T x0, BinaryOperator<T> op)` performs a reduction of `x0` and the elements using `op`. More precisely, writing `op.apply(x,y)` as infix `x`⊗`y`, returns `x0`⊗`x1`⊗`x2`⊗...⊗`xn`.
- `Collector<T,?,U> reducing(U e, Function<T,U> f, BinaryOperator<U> op)` performs a reduction of transformed elements using `op`. More precisely, writing `op.apply(x,y)` as infix `x`⊗`y`, returns an Optional containing the value `f.apply(x1)` ⊗ `f.apply(x2)` ⊗ ... ⊗ `f.apply(xn)` if the stream is non-empty; otherwise returns an empty Optional.
- `Collector<T,?,DoubleSummaryStatistics> summarizingDouble(ToDoubleFunction<T> f)` applies function `f` to each element and returns summary statistics for the resulting values; see section 24.5. There are similarly named methods for Int and Long.
- `Collector<T,?,Double> summingDouble(ToDoubleFunction<T> f)` applies function `f` to each element and returns the sum of the values. There are similarly named methods for Int and Long.
- `Collector<T,?,C> toCollection(Supplier<C> collectionFactory)` accumulates the elements into a new collection of type `<C extends Collection<T>>` created by the `collectionFactory`.
- `Collector<T,?,List<T>> toList()` accumulates the elements into a new list.
- `Collector<T,?,Map<K,U>> toMap(Function<T,K> fk, Function<T,U> fv)` accumulates the elements into a new map whose key-value pairs are `(fk.apply(xi), fv.apply(xi))`; throws IllegalStateException unless `fk.apply(xi)` is distinct for all elements `xi`.
- `Collector<T,?,Map<K,U>> toMap(Function<T,K> fk, Function<T,U> fv, BinaryOperator<U> merge)` accumulates the elements into a map whose key-value pairs are `(fk.apply(xi), xival)` where `xiVal` is the result `fv.apply(xi1)` ⊗ ... ⊗ `fv.apply(xin)` of merging the `fv`-values of all `xij` for which `fk.apply(xij)` equals `fk.apply(xi)`, where `x` ⊗ `y` is `merge.apply(x,y)`.

  There is also an overload with an additional argument of type `Supplier<Map<K,U>>` to produce the map used. There are also `toConcurrentMap` versions of the two preceding methods that produce concurrent maps, more efficient on parallel streams.
- `Collector<T,?,Set<T>> toSet()` accumulates the elements into a new set.

Example 184 Generating a Stream of Lists of Prime Factors
An infinite stream of lists of prime factors can be generated by mapping as suitable method `factorList` on the infinite stream $[2,3,\ldots]$. This is used in example 185.

```
public static List<Integer> factorList(int p) { ... }
public static Stream<List<Integer>> allFactorLists() {
  return IntStream.iterate(2, x -> x+1).mapToObj(Streams::factorList);
}
```

The computed lists of prime factors for the 11 numbers $2\ldots12$ look like this:

```
[2] [3] [2, 2] [5] [2, 3] [7] [2, 2, 2] [3, 3] [2, 5] [11] [2, 2, 3]
```

Example 185 Collectors for Grouping and Counting
Using a collector, one can group the lists of prime factors from example 184 by their length, that is, number `p` of prime factors. The grouping is represented by a map from `p` to the factor lists of length `p`, so group `1` contains the prime numbers, as shown below.

Moreover, instead of storing all factor lists, one can count them using another collector, to obtain a map from the prime factor count `p` to the number of numbers with that many prime factors; so `3=1273` below means that `1273` numbers between 2 and 5001 have `3` prime factors.

```
Map<Integer, List<List<Integer>>> factorGroups =
  allFactorLists().limit(11).collect(Collectors.groupingBy(List::size));
// {1=[[2], [3], [5], [7], [11]], 2=[[2, 2], [2, 3], [3, 3], [2, 5]], 3=[[2, 2, 2], [2, 2, 3]]}
Map<Integer, Long> factorGroupSizes =
  allFactorLists().limit(5000).collect(Collectors.groupingBy(List::size, Collectors.counting()));
// {1=669, 2=1366, 3=1273, 4=832, 5=452, 6=224, 7=104, 8=47, 9=22, 10=7, 11=3, 12=1}
```

Example 186 Grouping and Counting on a DoubleStream
Example 183 shows how to generate a van der Corput sequence. To test that the generated numbers are indeed evenly distributed over $[0,1]$ we put the numbers into 10 equally large bins and count them; there should be equally many numbers in each bin. We do this by calling the DoubleStream's three-argument `collect` method with a Supplier<`int[]`>, an ObjDoubleConsumer<`int[]`>, and a BiConsumer<`int[],int[]`> that directly collect the bin counts in a 10-element integer array. Generating and binning the first 100 million van der Corput numbers using this method takes 0.9 seconds using a single CPU core.

Alternatively, we could have used `.boxed()` to obtain a Stream<Double> and applied the one-argument `collect` method to a predefined collector, as in example 185. But the boxing of every `double` makes that approach slower by a factor of five.

```
final int bins = 10;
int[] binFrequenciesArray =
  vanDerCorput().limit(100_000_000).
  collect(() -> new int[bins],
          (a, x) -> { a[(int)(bins * x)]++; },
          (a1, a2) -> { for (int i=0; i<a1.length; i++) a1[i] += a2[i]; });
Arrays.stream(binFrequenciesArray).forEach(k -> System.out.printf("%d ", k));
// 10000002 10000001 10000000 10000000 9999997 10000002 10000001 10000000 10000000 9999997
```